1. Введение в bash

1.1 Оболочка

После входа в систему нас приветствует приглашение, которое как правило выглядит примерно как "$". Однако на практике оно также содержит имя хоста, имя текущей рабочей директории или все это вместе.
Программа, которая выводит это приглашение, называется оболочкой (shell). Чаще всего оболочкой в системе Linux является bash.

1.2 Запущен ли bash?

Для проверки того, запущена ли оболочка bash, можно использовать команду echo $SHELL.

```
┌──(kali㉿kali)-[~]
└─$ echo $SHELL
/usr/bin/zsh
```

Как видно из рисунка выше, у нас на машине кали, которая использовалась в первом модуле, стоит оболочка zsh, а не bash. Однако надо понимать, что в большинстве случаев, скрипты и команды, написанные на bash, будут работать в zsh. Это происходит потому что zsh поддерживает многие команды и синтаксис bash.
Также мы можем через терминал изменить оболочку для текущего пользователя. Но как видно на рисунке ниже, просто так оболочка меняться не хочет.

```
File  Actions  Edit  View
┌──(kali㉿kali)-[~]
└─$ chsh -s /bin/bash
Password:

┌──(kali㉿kali)-[~]
└─$ echo $SHELL
/usr/bin/zsh
```

Для изменения оболочки также необходимо перезагрузиться и вновь войти в систему, после чего оболочка будет успешно изменена.

```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ echo $SHELL
/bin/bash

┌──(kali㉿kali)-[~]
└─$
```

Как мы видим, мы действительно смогли изменить оболочку.

1.3 bash

Bash (Bourne again shell) - (еще одна командная оболочка Борна, то есть рожденная вновь командная оболочка) - оболочка по умолчанию в большинстве современных систем Linux.
Задача оболочки заключается в том, чтобы получать от пользователя команды, через которые мы взаимодействуем с системой.
После ввода команды мы модем выйти из оболочки по команде exit (ну то есть по сути мы закрываем так терминал). Также можно не выйти, а закончить сеанс (logout).

```
┌──(root㉿kali)-[/home/kali]
└─# exit

┌──(kali㉿kali)-[~]
└─$
```

Также выйти из оболочки можно при помощи сочетания клавиш Ctrl+D.
Используя команду cd / , мы переместимся в корневую папку.

```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ cd /

┌──(kali㉿kali)-[/]
└─$
```

1.4 Пути

Для того, чтобы узнать в какой директории сейчас мы находимся можно использоваться команду pwd.

```
┌──(kali㉿kali)-[/]
└─$ pwd
/
```

```
┌──(kali㉿kali)-[~]
└─$ pwd
/home/kali
```

1.5 Абсолютные пути

Примеры абсолютных путей:

- /dev
- /usr
- /usr/bin
- /usr/local/bin
  Все абсолютные пути начинаются с "/". Это из-за того, что абсолютные пути всегда прописываются начиная с корня.

1.6 Относительные пути

Относительные пути никогда не начинаются со знака "/". Они рассматриваются уже не относительно корня, а относительно текущей директории.

```
┌──(kali㉿kali)-[~]
└─$ cd /usr

┌──(kali㉿kali)-[/usr]
└─$ cd local/bin

┌──(kali㉿kali)-[/usr/local/bin]
└─$ pwd
/usr/local/bin
```

Также относительный путь может содержать "..", что является указанием на родительскую директорию.

```
┌──(kali㉿kali)-[/usr/local/bin]
└─$ cd ..

┌──(kali㉿kali)-[/usr/local]
└─$ pwd
/usr/local
```

Как видно на рисунке выше, мы переместились на один уровень выше.
Аналогично можно переместиться в соседнюю директорию:

```
┌──(kali㉿kali)-[/usr/local]
└─$ cd ../share

┌──(kali㉿kali)-[/usr/share]
└─$ pwd
/usr/share
```

1.7 Примеры относительных путей

Относительные пути могут быть логически и символьно более сложными. Например такие, как на рисунке ниже.

```
$ cd /bin
$ cd ../usr/share/zoneinfo

$ cd /usr/X11R6/bin
$ cd ../lib/X11

$ cd /usr/bin
$ cd ../bin/../bin
```

Набрав команды у себя, мы увидим, куда будут вести эти пути.

```
┌──(kali㉿kali)-[/usr/share]
└─$ cd /bin

┌──(kali㉿kali)-[/bin]
└─$ cd ../usr/share/zoneinfo

┌──(kali㉿kali)-[/usr/share/zoneinfo]
└─$ pwd
/usr/share/zoneinfo
```

```
┌──(kali㉿kali)-[~]
└─$ cd /usr/X11R6/bin
bash: cd: /usr/X11R6/bin: No such file or directory

┌──(kali㉿kali)-[~]
└─$ cd /usr/X11R6/bin cd ../lib/X11
bash: cd: too many arguments

┌──(kali㉿kali)-[~]
└─$ cd ../lib/X11
bash: cd: ../lib/X11: No such file or directory

┌──(kali㉿kali)-[~]
└─$ cd /lib/X11

┌──(kali㉿kali)-[/lib/X11]
└─$ cd /usr/bin
```

```
┌──(kali㉿kali)-[/usr/bin]
└─$ cd ../bin/../bin

┌──(kali㉿kali)-[/usr/bin]
└─$
```

1.8 Еще одна спецдиректория "."

Директория "." означает текущую директорию. Она редко используется при использовании команды cd, однако ее можно увидеть в других случаях. Например когда нам надо выполнить программу из текущей директории по команду ./myprog.

1.9 cd и домашняя директория

Чтобы переместиться в домашнюю директорию достаточно просто использовать cd без аргументов.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ cd

┌──(kali㉿kali)-[~]
└─$ pwd
/home/kali

┌──(kali㉿kali)-[~]
└─$
```

Вспомним вышеприведенный пример ./myprog. Предположим, что нам надо подать в

эту программу в качестве аргумента путь к файлу. Сделать мы это можем двумя способами:

- ./myprog /home/droppins/myfile.txt
- ./myprog ~/myfile.txt, где "~" будет эквивалентно адресу домашней директории

Другие домашние директории пользователей

bash воспринимает "~" как указание на нашу домашнюю директорию. Однако также можно указывать и на домашние каталоги других пользователей, например: ./myprog ~fred/fredsfile.txt

2. Использование команд Linux

2.1 Знакомство с ls

Мы уже знаем, что используя команду ls можно посмотреть какие файлы есть в текущей директории.

```
┌──(kali㉿kali)-[~]
└─$ ls
Desktop    Downloads  impacket   Music      Pictures  pypykatz  Templates
Documents  hash.txt   lsass.DMP  neofetch   Public    temp      Videos

┌──(kali㉿kali)-[~]
└─$
```

Теперь также введем ls, но уже с опцией -a, что позволит нам видеть все файлы, включая скрытые файлы и директории, которые начинаются с ".".

```
┌──(kali㉿kali)-[~]
└─$ ls -a
.                   .pki
..                  .profile
.bash_history       Public
.bash_logout        pypykatz
.bashrc             .sudo_as_admin_successful
.bashrc.original    temp
.cache              Templates
.config             .vboxclient-clipboard-tty7-control.pid
Desktop             .vboxclient-clipboard-tty7-service.pid
.dmrc               .vboxclient-display-svga-x11-tty7-control.pid
Documents           .vboxclient-display-svga-x11-tty7-service.pid
Downloads           .vboxclient-draganddrop-tty7-control.pid
.face               .vboxclient-draganddrop-tty7-service.pid
.face.icon          .vboxclient-hostversion-tty7-control.pid
.gnupg              .vboxclient-seamless-tty7-control.pid
hash.txt            .vboxclient-seamless-tty7-service.pid
.ICEauthority       .vboxclient-vmsvga-session-tty7-control.pid
impacket            Videos
.java               .viminfo
.john               .Xauthority
.local              .xsession-errors
lsass.DMP           .xsession-errors.old
.mozilla            .zprofile
Music               .zsh_history
neofetch            .zshrc
Pictures

┌──(kali㉿kali)-[~]
└─$
```

Как видно, выводятся не только файлы и директории, начинающиеся с ".", но и с "..".

Развернутые списки директорий

Также можно задать одну и более директорий или файлов с командой ls. Если указать в ls только файл, то отобразится только он. Если же задать директорию, то отобразится все ее содержимое.

Используя опцию -l (No_Shift+L), можно просматривать права доступа на файлы и директории.

```
┌──(kali㉿kali)-[~]
└─$ ls -l /usr
total 216
drwxr-xr-x   2 root root 126976 Jul  3 12:08 bin
drwxr-xr-x   2 root root   4096 May 15 21:57 games
drwxr-xr-x   5 root root   4096 May 29 15:13 i686-w64-mingw32
drwxr-xr-x  82 root root  20480 Jul  3 11:05 include
drwxr-xr-x 118 root root   4096 Jul  3 12:08 lib
drwxr-xr-x   3 root root   4096 Jul  3 11:05 lib32
drwxr-xr-x   2 root root   4096 Jul  3 11:05 lib64
drwxr-xr-x  23 root root   4096 Jun 21 06:24 libexec
drwxr-xr-x  11 root root   4096 May 29 15:02 local
drwxr-xr-x   2 root root  20480 Jul  3 11:06 sbin
drwxr-xr-x 360 root root  12288 Jul  3 12:08 share
drwxr-xr-x   3 root root   4096 May 29 15:16 src
drwxr-xr-x   5 root root   4096 May 29 15:13 x86_64-w64-mingw32

┌──(kali㉿kali)-[~]
└─$
```

Как видно из рисунка выше, первая колонка отображает права доступа, вторая колонка содержит количество ссылок на каждый элемент файловой системы. Третья колонка - это список владельцев. Четвертая колонка - список групп. Пятая колонка отображает размер объектов. Шестая колонка показывает время последнего изменения, а седьмая отображает имя файла.

Иногда может возникать ситуация, когда надо рассмотреть директорию, не заходя в нее. В таком случае можно использовать ls с опцией -dl, при помощи которой ls рассмотрит любую директорию так, будто она внутренняя.

```
┌──(kali㉿kali)-[~]
└─$ ls -dl /usr /usr/bin
drwxr-xr-x 15 root root   4096 May 29 15:17 /usr
drwxr-xr-x  2 root root 126976 Jul  3 12:08 /usr/bin

┌──(kali㉿kali)-[~]
└─$
```

Рекурсивный и инодный списки

Для того, чтобы рекурсивно посмотреть на директорию и файлы внутри нее, а также на состав директорий внутри текущей, можно использовать опцию -R. Однако надо иметь в виду, что вывод такой команды чаще всего очень большой.

Опция -i может быть использована для отображения числа инодов для объектов в списке файловой системы.

```
┌──(kali㉿kali)-[~]
└─$ ls -i
3276851 Desktop    3281931 hash.txt   3276856 Music     3276854 Public    3276853 Templates
3276855 Documents  3278048 impacket   3281929 neofetch  3278275 pypykatz  3276858 Videos
3276852 Downloads  3277993 lsass.DMP  3276857 Pictures  3276909 temp

┌──(kali㉿kali)-[~]
└─$
```

Каждый объект в файловой системе имеет собственный уникальный индекс, который называется номером инода.

Для примера рассмотрим ссылки "." и "..". Для начала в таком случае посмотрим номер инода для /usr/local.

```
┌──(kali㉿kali)-[~]
└─$ ls -id /usr/local
1217137 /usr/local

┌──(kali㉿kali)-[~]
└─$
```

Так теперь мы знаем, что у /usr/local номер инода равен 1217137. Теперь посмотрим на /usr/local/bin.

```
┌──(kali㉿kali)-[~]
└─$ ls -id /usr/local
1217137 /usr/local

┌──(kali㉿kali)-[~]
└─$ ls -id /usr/local/bin
1217171 /usr/local/bin
```

Как видно, у них совпадает часть 12171 и не совпадают последние 2 цифры, что говорит о том, что они отсылают друг к другу в соответствии с иерархией.
Если еще раз взглянуть на /usr/local через ls -al, то мы увидим, что на него (на его инод) в системе есть 11 ссылок.

```
┌──(kali㉿kali)-[~]
└─$ ls -al  /usr/local
total 44
drwxr-xr-x 11 root root 4096 May 29 15:02 .
drwxr-xr-x 15 root root 4096 May 29 15:17 ..
drwxr-xr-x  2 root root 4096 May 29 15:02 bin
drwxr-xr-x  2 root root 4096 May 29 15:02 etc
drwxr-xr-x  2 root root 4096 May 29 15:02 games
drwxr-xr-x  2 root root 4096 May 29 15:02 include
drwxr-xr-x  4 root root 4096 May 29 15:12 lib
drwxr-xr-x  2 root root 4096 May 29 15:02 libexec
lrwxrwxrwx  1 root root    9 May 29 15:02 man → share/man
drwxr-xr-x  2 root root 4096 Jun 15 06:43 sbin
drwxr-xr-x  9 root root 4096 May 29 15:19 share
drwxr-xr-x  2 root root 4096 May 29 15:02 src

┌──(kali㉿kali)-[~]
└─$
```

2.2 mkdir

Команда mkdir нужна для создания новой директории. Для примера создадим 3 новых директории в папке /tmp.

```
┌──(kali㉿kali)-[~]
└─$ cd /tmp

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2

┌──(kali㉿kali)-[/tmp]
└─$ mkdir tia tac toe

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe

┌──(kali㉿kali)-[/tmp]
└─$
```

По умолчанию mkdir не создает родительские директории. Это значит, что при создании папки весь путь вплоть до последнего должен существовать. В ином случае потребуется сделать несколько операций вместо одной.

```
┌──(kali㉿kali)-[/tmp]
└─$ mkdir hell/low
mkdir: cannot create directory ‘hell/low’: No such file or directory

┌──(kali㉿kali)-[/tmp]
└─$ mkdir hell

┌──(kali㉿kali)-[/tmp]
└─$ mkdir hell/low

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
hell
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe

┌──(kali㉿kali)-[/tmp]
└─$ ls hell
low

┌──(kali㉿kali)-[/tmp]
└─$
```

Однако также у mkdir есть опция -p, которая дает возможность команде создавать любые отсутствующие родительские директории. Например:

```
┌──(kali㉿kali)-[/tmp]
└─$ mkdir -p try/make/this

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
hell
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe
try

┌──(kali㉿kali)-[/tmp]
└─$ ls try -R
try:
make

try/make:
this

try/make/this:

┌──(kali㉿kali)-[/tmp]
└─$
```

2.3 touch

- cp - копирование файла
- mv - перемещение и переименование файла
  Для начала используем команду cd /tmp | touch copyme. Как видно на рисунке ниже, поскольку файла copyme ранее не было, он был создан.

```
┌──(kali㉿kali)-[/tmp]
└─$ cd /tmp | touch copyme

┌──(kali㉿kali)-[/tmp]
└─$ cd

┌──(kali㉿kali)-[~]
└─$ cd /tmp

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
copyme
hell
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe
try

┌──(kali㉿kali)-[/tmp]
└─$
```

У этого файла сейчас нулевой размер и он пуст.

2.4 echo

Продолжая пример из предыдущего подраздела, добавим в новоиспеченный файл copyme немного данных. Это можно сделать при помощи команды ehco.
Если ее использовать без опций, то текст будет выведен просто не стандартный вывод.

```
┌──(kali㉿kali)-[/tmp]
└─$ echo "firstfile"
firstfile
```

Однако также мы можем и перенаправить вывод. Тогда команда будет иметь следующий вид: echo "firstfile" > copyme.

```
┌──(kali㉿kali)-[/tmp]
└─$ echo "firstfile"
firstfile

┌──(kali㉿kali)-[/tmp]
└─$ echo "firstfile" > copyme

┌──(kali㉿kali)-[/tmp]
└─$ cat copyme
firstfile

┌──(kali㉿kali)-[/tmp]
└─$
```

Как видно из рисунка выше, мы действительно переместили эту фразу в copyme, после чего при помощи cat считали текст из copyme и увидели искомую фразу.
Если бы файл copyme не существовал к моменту запуска команды echo, то при обращении к несуществующему файлу, echo создала бы его.
Далее посмотрим более детальную информацию об измененном файле при помощи команды ls -l.

```
┌──(kali㉿kali)-[/tmp]
└─$ ls -l copyme
-rw-rw-r-- 1 kali kali 10 Jul 13 06:54 copyme
```

Как видно, теперь он весит 10 байт, тк содержит firstfile и символ новой строки.
Теперь попробуем скопировать файл при помощи cp.

```
┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
copiedme
copyme
hell
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe
try

┌──(kali㉿kali)-[/tmp]
└─$ cat copiedme
firstfile

┌──(kali㉿kali)-[/tmp]
└─$
```

Как видно, файл скопирован и почти полностью идентичен оригиналу. Однако еще стоит проверить иноды. Для этого используем команду ls -i.

```
┌──(kali㉿kali)-[/tmp]
└─$ ls -i copyme copiedme
77 copiedme  75 copyme

┌──(kali㉿kali)-[/tmp]
└─$
```

Как мы можем увидеть на рисунке выше, иноды не совпадают, что подтверждает факт того, что это разные файлы.

2.5 mv

Используем mv для переименования файла copiedme. При этом номер инода останется прежним.

```
┌──(kali㉿kali)-[/tmp]
└─$ mv copiedme movedme

┌──(kali㉿kali)-[/tmp]
└─$ ls
config-err-geEuDG
copyme
hell
movedme
ssh-u6r9iShS2Kzb
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-colord.service-MBFvN0
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-haveged.service-P6QqNg
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-ModemManager.service-btjCCP
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-polkit.service-THsSEp
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-systemd-logind.service-jkCgSr
systemd-private-8e20edc3c6b844eb8d5d10f1d776c37e-upower.service-ZuTtT2
tac
tia
toe
try

┌──(kali㉿kali)-[/tmp]
└─$
```

```
┌──(kali㉿kali)-[/tmp]
└─$ ls -i movedme
77 movedme

┌──(kali㉿kali)-[/tmp]
└─$
```

Номер инода остается прежним, пока путь к файлу остается прежним.
Также mv можно использовать для перемещения файла, например mv /var/tmp/myfile.txt /home/droppins, что перемести файл myfile из var/tmp в home/droppins. В таком случае номер инода этого файлы изменится.
Аналогично можно переместить несколько файлов сразу: mv /var/tmp/myfile1.txt /var/tmp/myarticle3.txt /home/drobbins

3. Создание ссылок и удаление файлов

3.1 Жесткие ссылки

В Linux есть 2 типа ссылок: жесткие и символьные.

Жесткие - работают на уровне инодов для указания конкретного файла. Для жестких ссылок есть ограничения: жесткие ссылки создаются только на файла, не на директории; нельзя связать жесткими ссылками несколько файловых систем.

Создать новые жесткие ссылки можно при помощи ln.

3.2 Символьные ссылки

Символьные (символические) ссылки, они же симлинки, используются на практике чаще, чем жесткие.
Симлинки - это файлы особого типа, которые ссылаются на другие файлы по имени, а не прямо по номеру инода. Они не спасают файл от удаления - если файл, на который указывает ссылка, исчезнет, то симлинк лоломается.

Создать символьные ссылки можно при помощи ln -s.

При выводе ls -l символьные ссылки можно отличить тремя способами:

1. посмотреть на символ l (No-Shift+L) в первой колонке
2. размер символической ссылки равен количеству символов в ней
3. последняя колонка в выводе указывает на то, куда ведет ссылка с помощью обозначения "->"

В целом символьные ссылки более гибкие, чем жесткие, на то они и символьные. Можно создать символьную ссылку на любой объект файловой системы, в том числе на директории. Благодаря тому, что их реализация основана на путях, а не на инодах, мы можем свободно создавать символьные ссылки, указывающие на объект другой файловой системы.

Например создадим ссылку в /tmp с указанием на /usr/local/bin.

```
┌──(kali㉿kali)-[/tmp]
└─$ ln -s /usr/local/bin bin1

┌──(kali㉿kali)-[/tmp]
└─$ ls -l bin1
lrwxrwxrwx 1 kali kali 14 Jul 13 07:31 bin1 → /usr/local/bin

┌──(kali㉿kali)-[/tmp]
└─$
```

Также сделаем еще один вариант такой ссылки, но немного иначе.

```
┌──(kali㉿kali)-[/tmp]
└─$ ln -s ../usr/local/bin bin2

┌──(kali㉿kali)-[/tmp]
└─$ ls -l bin2
lrwxrwxrwx 1 kali kali 16 Jul 13 07:32 bin2 → ../usr/local/bin

┌──(kali㉿kali)-[/tmp]
└─$
```

Как мы видим, они указывают на одну директорию, но вторая символьная ссылка может поломаться при перемещении в другую директорию из-за того, что в отличие от первой ссылки (bin1), здесь (bin2) используется относительный путь.

Таким образом, в некоторых случаях будет лучше избегать использования относительных путей при создании ссылок.
Создадим от имени рута синоним для keychain, то есть kc.

```
┌──(root㉿kali)-[/usr/bin]
└─# ln -s /usr/bin/keychain kc
```

```
└─# ls -l kc
lrwxrwxrwx 1 root root 17 Jul 13 07:41 kc → /usr/bin/keychain
```

Однако если мы сейчас переместим оба файла, то ссылка перестанет работать, потому что искомый путь будет изменен.

3.3 rm

Теперь попробуем удалять файлы. Для этого нам понадобится команда rm.
мало кто знает, на запускать команду sudo rm -rf /* немного плохая идея.
Для примера создадим 2 файла, после чего удалим их.

```
┌──(kali㉿kali)-[~]
└─$ cd /tmp

┌──(kali㉿kali)-[/tmp]
└─$ touch file1 file2

┌──(kali㉿kali)-[/tmp]
└─$ ls -l file1 file2
-rw-rw-r-- 1 kali kali 0 Jul 13 08:09 file1
-rw-rw-r-- 1 kali kali 0 Jul 13 08:09 file2

┌──(kali㉿kali)-[/tmp]
└─$ rm file1 file2

┌──(kali㉿kali)-[/tmp]
└─$ ls -l file1 file2
ls: cannot access 'file1': No such file or directory
ls: cannot access 'file2': No such file or directory

┌──(kali㉿kali)-[/tmp]
└─$
```

Иногда имеет смысл использовать опцию -i для команды rm, благодаря которой будет

запрашиваться подтверждение на удаление. Это важно, потому что в таком случае файлы удаляются не переносом в корзину, а практически безвозвратно.

```
┌──(kali㉿kali)-[/tmp]
└─$ rm -i file1
rm: remove regular empty file 'file1'? y
```

Для удаления директории есть 2 варианта действий:

1. использовать команду rmdir

```
┌──(kali㉿kali)-[/tmp]
└─$ mkdir mydir

┌──(kali㉿kali)-[/tmp]
└─$ touch mybir/file1
touch: cannot touch 'mybir/file1': No such file or directory

┌──(kali㉿kali)-[/tmp]
└─$ touch mydir/file1

┌──(kali㉿kali)-[/tmp]
└─$ rm mydir/file1

┌──(kali㉿kali)-[/tmp]
└─$ rmdir mydir

┌──(kali㉿kali)-[/tmp]
└─$
```

2. рекурсивное удаление через опцию -rf (recoursive force) команды rm.

```
┌──(kali㉿kali)-[/tmp]
└─$ mkdir mydir

┌──(kali㉿kali)-[/tmp]
└─$ touch mydir/file1

┌──(kali㉿kali)-[/tmp]
└─$ rm -rf mydir

┌──(kali㉿kali)-[/tmp]
└─$
```

4. Использование джокеров

4.1 Знакомство с джокерами

Может быть ситуация, когда надо удалить много файлов разом, а прописывание каждого из них в командную строку будет долгим и нудным. Тогда можно использовать встроенную в Linux поддержку джокеров (globbing, то есть универсализация файловых имен). Это позволит указать много файлов за раз, используя шаблон "дикой конкатенации". Это работает так, что bash и другие команды Linux просматривают диск в поисках файлов, подходящих под шаблон.
Например, если нам надо удалить файлы file1, file2, ... file8, то можно использовать

команду rm file[1-8].
Если нам надо удалить все файлы, имя которых начинается с file, то можно использовать команду rm file.
Дикая конкатенация " " совпадает с любым символом или набором символов, а также с их отсутствием.

4.2 Неподходящие шаблоны

Если нам нужно перечислить все объекты файловой системы в /etc, которые начинаются на "g", то можно использовать ls -d /ets/g.

В случае же, если искомый шаблон не подойдет ни под один объект файловой системы, то появится ошибка, говорящая, что такого файла нет.
Основное правило: glob-шаблоны разворачиваются, только если совпадают в объектами файловой системы.
Спецсимволы, которые используются в качестве джокеров:

- - - совпадает с нулевым или большим количеством символов
- ? - равен любом одному символу
- [] - в скобки помещаются все символы, которые нужны. Также можно указать диапазон через "-", например 1-8
- [!] - обратное от просто пустых скобок. Фактически означает "все, кроме [!]"

Предостережения о джокерах

Вногда лучше указать более конкретно что именно надо внести в файл, например вместо echo [fo*] > /tmp/mynewfile.txt лучше написать echo '[fo*]' > /tmp/mynewfiel или echo \ \ [fo \ \ ] \ \ *

5. Регулярные выражения

Регулярное выражение (regular expression, regexp, regex) - особый синтаксис, используемый для описания текстовых шаблонов. В Linux-системах регулярные выражения используются для поиска в тексте по шаблону, а также для операций поиска и замены на текстовых потоках.

Синтаксис регулярных выражений похож на синтаксис подстановки имен файлов (globbing). Однако эта схожесть является лишь поверхностной.

5.1 Простая подстрока

Для рассмотрения простой подстроки используем команду grep, которая позволяет сканировать содержимое файла согласно заданному регулярному выражению.
Команды grep выводит каждую строчку, которая совпадает с регулярным выражением, игнорируя остальные.

```
┌──(kali㉿kali)-[/tmp]
└─$ grep bash /etc/passwd
postgres:x:126:130:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
kali:x:1000:1000::/home/kali:/bin/bash
sshuser:x:1001:1001:userr,1,,:/home/sshuser:/bin/bash

┌──(kali㉿kali)-[/tmp]
└─$
```

В случаях, когда подстрока должна содержать "+", ".", " * ", "[]" или ",", то эти символы должны быть экранированы обратным слешем, а сама подстрока должна быть в кавычках.

5.2 Метасимволы

При помощи регулярных выражений можно осуществлять гораздо более сложный поиск, чем в примерах выше, если использовать метасимволы. Например "." совпадает с любым единичным символом, то есть точно также как "?" в glob-подстановках.

Если нам надо задать более конкретно символ, чем ".", то можно использовать "[ ]", чтобы указать подмножество символов сопоставления.

Можно использовать "[^]", чтобы обозначить символы, которые не надо искать, то есть тоже, что [!] в glob-подстановках.

5.3 Синтаксис

Синтаксис внутри квадратных скобок сильно отличается от синтаксиса остальной части регулярного выражения. Например "." в квадратных скобках интерпретируется однозначно в качестве точки, а вне скобок - как метасимвол, если не приставить к ней кавычки.

5.4 Метасимвол *

Символ " * " используется для сопоставления нулевому или большему числу повторений предыдущего символа, если он стоит в паре с каким-то символом перед ним.
Например:

- - ab*c совпадает с «abbbbc», но не с «abqc» (в случае glob-подстановки, обе строчки будут удовлетворять шаблону. Вы уже поняли почему?)
- ab*c совпадает с «abc», но не с «abbqbbc» (опять же, при glob-подстановке, шаблон сопоставим с обоими строчками)
- ab*c совпадает с «ac», но не с «cba» (в случае глоббинга, ни «ac», ни «cba» не удовлетворяют шаблону)
- b[cq]*e совпадает с «bqe» и с «be» (glob-подстановке удовлетворяет «bqe», но не «be»)

- b[cq]*e совпадает с «bccqqe», но не с «bccc» (при глоббинге шаблон точно так же совпадет с первым, но не со вторым)
- b[cq]*e совпадает с «bqqcce», но не с «cqe» (так же и при glob-подстановке)
- b[cq]*e удовлетворяет «bbbeee» (но не в случае глоббинга)
- .* сопоставим с любой строкой (glob-подстановке удовлетворяют только строки начинающиеся с ".")
- foo.* совпадет с любой подстрокой начинающийся с «foo» (в случае glob-подстановки этот шаблон будет совпадать со строками, начинающимися с четырех символов «foo.»)

5.5 Начало и конец строки

Символы "^" и "$" используются для обозначения начала и конца подстроки.

- используя ^, мы прикрепим шаблон к началу строки
- используя $, мы прикрепим шаблон к концу строки

5.6 Полнострочные регулярные выражения

"^" и " $ " можно комбинировать для сопоставлений со всей строкой целиком, например grep '^#.* \ \ . $' /etc/fstab

6. FHS и поиск файлов

6.1 Стандарт иерархии файловой системы

Стандарт иерархии файловой системы (Filesystem Hierarchy Standard (FHS)) - документ, определяющий схему директорий в Linux-системах. FHS был разработан для того, чтобы представить общую схему для упрощения независимой от дистрибутива разработки ПО. Дело в том, что все необходимое располагается одинаково в большей части дистрибутивов.

Древо директорий, в соответствии с FHS:

- / - корень
- /boot - статичные файлы загрузчика
- /dev - файлы устройств
- /etc - специфические для хоста файлы конфигурации
- /lib - основные разделяемые библиотеки и модули ядра
- /mnt - точка монтирования для съемных носителей
- /opt - дополнительные пакеты ПО
- /sbin - основные системные программы
- /tmp - временные файлы
- /usr - вторичная иерархия

- /var - изменяемые данные

Две независимые классификации в FHS

Существует 2 независимые классификации файлов: изменяемые и неизменяемые (статичные) , разделяемые и неразделяемые. Разделяемые данные могут распределяться на несколько хостов, а неразделяемые специфичны для конкретного хоста. Изменяемые данные могут изменяться, а статичные не изменяются.

	разделяемые	**неразделяемые**
статичные	/usr, /opt	/etc, /boot
изменяемые	/var/mail, /var/spool/news	/var/run, /var/lock

6.2 Вторичная иерархия /usr

Внутри /usr находится вторичная иерархия, которая очень похожа на корневую файловую систему. Для /usr не критично существование во время включения машины, она может быть сетевым ресурсом (разделяема) или примонтирована на CD-ROM (статична).
Большая часть дистрибутивов не используют разделяемость /urs, но важно понимать полезность отличия между основной иерархией и вторичной.

7. Поиск файлов

7.1 PATH

Когда мы запускаем программу с помощью командной строки, bash начинает просматривать список директорий в поисках программы, которую мы указали. Например, когда мы вводим ls, bash не знает заранее, что программа ls находится в /usr/bin. Поэтому он ссылается на переменную окружения PATH, которая содержит список директорий разделенных двоеточием.
Мы можем собственноручно проверить значение PATH:

```
┌──(kali㉿kali)-[/tmp]
└─$ echo $PATH
/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:
/home/kali/.dotnet/tools

┌──(kali㉿kali)-[/tmp]
└─$
```

Переменную PATH можно расширять, присваивая ей новое значение:

```
┌──(kali㉿kali)-[/tmp]
└─$ echo $PATH
/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:
/home/kali/.dotnet/tools

┌──(kali㉿kali)-[/tmp]
└─$ PATH=$PATH:~/bin

┌──(kali㉿kali)-[/tmp]
└─$ echo $PATH
/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:
/home/kali/.dotnet/tools:/home/kali/bin

┌──(kali㉿kali)-[/tmp]
└─$
```

Можно и удалять элементы их PATH. Однако это будет немного сложнее, тк мы не можем ссылаться в команде на существующий $PATH. Самый простой и удобный варианта - просто заново указать в PATH то, что нужно нам:

```
┌──(kali㉿kali)-[/tmp]
└─$ echo $PATH
/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:
/home/kali/.dotnet/tools:/home/kali/bin

┌──(kali㉿kali)-[/tmp]
└─$ PATH=/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/home/kali/.d
otnet/tools:/home/kali/bin

┌──(kali㉿kali)-[/tmp]
└─$ echo $PATH
/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/home/kali/.dotnet/too
ls:/home/kali/bin

┌──(kali㉿kali)-[/tmp]
└─$
```

Если мы хотим, чтобы наши изменения PATH были доступны для процессов, которые будут запускаться в командной оболочке, то надо "экспортировать" их при помощи команды export PATH.

Команда which

Команда which позволяет проверить, есть ли конкретная программа в нашем PATH.

```
┌──(kali㉿kali)-[/tmp]
└─$ which sense

┌──(kali㉿kali)-[/tmp]
└─$ which ls
/usr/bin/ls
```

Команда sense не найдена, потому что мы ее не использовали, а вот ls мы использовали.
Также можно использовать опцию -a, которая укажет which показать все экземпляры программы в PATH.

```
┌──(kali㉿kali)-[/tmp]
└─$ which -a ls
/usr/bin/ls
/bin/ls
```

В случае, когда нам нужно больше информации о программе, чем просто расположение, можно использовать команду whereis.

```
┌──(kali㉿kali)-[/tmp]
└─$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz

┌──(kali㉿kali)-[/tmp]
└─$
```

В целом программу whereis можно использовать для нестандартного поиска и поиска расположения исходников. Также можно указать этой программе альтернативные пути для поиска.

Команда find

Команда find - это еще один удобный инструмент для поиска. При использовании find мы можем искать не только программы. Можно искать любые типы файлов, используя при этом различные критерии для поиска.
Например, попробуем найти файлы в директории /usr/share/doc, которые называются README.

```
┌──(kali㉿kali)-[/tmp]
└─$ find /usr/share/doc -name README
/usr/share/doc/libafflib0t64/README
/usr/share/doc/smartmontools/README
/usr/share/doc/smartmontools/examples/README
/usr/share/doc/python3-pcapy/README
/usr/share/doc/libiec61883-0/README
/usr/share/doc/sound-icons/README
/usr/share/doc/libdatrie1/README
/usr/share/doc/redis-tools/examples/lru/README
/usr/share/doc/libwnck-3-0/README
/usr/share/doc/urlscan/README
/usr/share/doc/fakeroot/README
/usr/share/doc/lrzsz/README
/usr/share/doc/libslang2/README
/usr/share/doc/libunicode-linebreak-perl/README
/usr/share/doc/libavahi-common-data/README
/usr/share/doc/avahi-utils/README
/usr/share/doc/ncrack/examples/README
```

Как видно, нашлось таких много.

Можно использовать glob-шаблоны для аргументов -name, при условии, что мы экранируем их кавычками или обратным слешем.

Можно игнорировать регистр при поиске. Если вернуться к поиску файла README, то вот 2 варианта, как это сделать:

- ```
  $ find /usr/share/doc -name '[Rr][Ee][Aa][Dd]
  [Mm][Ee]*'
  ```
  ```

- ```
  $ find /usr/share/doc -iname readme*
  ```

Для поиска без учета регистра можно использовать опцию -iname.
Для поиска по шаблону можно использовать опцию -regex и -iregex (без учета регистра поиск).
find требует, чтобы регулярное выражение указывалось для всего пути, а не только его части. Это значит, что надо в начале и в конце шаблона ставить ".* "; простого использования xt в качестве шаблона будет недостаточно.
При помощи опции -type можно искать в файловой системе файлы определенного типа.

Возможные аргументы для -type:

- b - блочное устройство
- c - символьное устройство
- d - директория
- p - именованный канал
- f - обычный канал
- l (No_Shift+L) - символьная ссылка
- s - сокет

Например, вот так будет выглядеть команда для поиска символической ссылки в /usr/bin, которая содержит строку vim в своем имени: find /usr/bin -name ' *vim* ' -type l.

```
┌──(kali㉿kali)-[/tmp]
└─$ find /usr/bin -name '*vim*' -type l
/usr/bin/vimdiff
/usr/bin/rvim
/usr/bin/vim

┌──(kali㉿kali)-[/tmp]
└─$
```

При помощи опции -mtime можно искать файлы, основываясь на дате их последней модификации. Аргументом в опции -mtime выступает количество 24-чаосвых периодов, и полезным будет указывать перед аргументом плюс (+, означает "после") или минус (-, означает "перед").
Например:
```

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ ls -l
total 4
-rw-rw-r-- 1 kali kali 11 Jul 13 10:37 a
-rw-rw-r-- 1 kali kali  0 Jul 13 10:34 b
-rw-rw-r-- 1 kali kali  0 Jul 13 10:34 c
-rw-rw-r-- 1 kali kali  0 Jul 13 10:34 d
```

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ find . -name \? -mtime -1
.
./d
./c
./b
./a
```

При помощи опции -daystart можно сделать так, чтобы периоды времени отсчитывались от начала сегодняшнего дня, а не от текущего времени.

При помощи опции -size можно искать файлы по их размеру. По умолчанию аргументом для size является количество 512-байтных блоков, но добавляя к опции нужный суффикс. можно сделать все более понятным.
Суффиксы для -size:

- b - 512-байтные блоки
- c - байты
- k - килобайты
- w - 2-байтные слова

Также перед аргументом можно поставить + (больше чем) или - (меньше чем).
Например:

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ find /usr/bin -type f -size -50c
/usr/bin/gophish-stop
/usr/bin/7za
/usr/bin/mpg123-oss
/usr/bin/rgrep
/usr/bin/pnmtoplainpnm
/usr/bin/zegrep
/usr/bin/egrep
/usr/bin/fgrep
/usr/bin/7zr
/usr/bin/7z
/usr/bin/zfgrep
/usr/bin/mpg123-nas
```

7.2 Работа с найденными файлами

При помощи опции -exec команда find может производить любые действия с файлами. Эта опция принимает для выполнения строку команд, которая оканчивается на ;, и

заменяет все вхождения {} именем файла.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ find /usr/bin -type f -size -50c -exec ls -l '{}' ';'
-rwxr-xr-x 1 root root 43 May 30  2023 /usr/bin/gophish-stop
-rwxr-xr-x 1 root root 39 May  8 06:46 /usr/bin/7za
-rwxr-xr-x 1 root root 49 Dec 22  2024 /usr/bin/mpg123-oss
-rwxr-xr-x 1 root root 30 Jul 21  2023 /usr/bin/rgrep
-rwxr-xr-x 1 root root 31 Apr 12 05:24 /usr/bin/pnmtoplainpnm
-rwxr-xr-x 1 root root 29 Jan 16 23:18 /usr/bin/zegrep
-rwxr-xr-x 1 root root 41 Jan  4  2024 /usr/bin/egrep
-rwxr-xr-x 1 root root 41 Jan  4  2024 /usr/bin/fgrep
-rwxr-xr-x 1 root root 39 May  8 06:46 /usr/bin/7zr
-rwxr-xr-x 1 root root 38 May  8 06:46 /usr/bin/7z
-rwxr-xr-x 1 root root 29 Jan 16 23:18 /usr/bin/zfgrep
-rwxr-xr-x 1 root root 49 Dec 22  2024 /usr/bin/mpg123-nas

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

7.3 locate

Команде find может потребоваться некоторое время на выполнение, тк ей нужно прочитать каждую директорию, где выполняется поиск. Команда locate может ускорить этот процесс используя внешнюю базу данных, генерируемую updatedb.
Команда locate будет искать совпадения любой части пути, а не только самого файла. Например:

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ locate /bin/ls
/usr/bin/ls
/usr/bin/lsassy
/usr/bin/lsattr
/usr/bin/lsb_release
/usr/bin/lsblk
/usr/bin/lscpu
/usr/bin/lsfd
/usr/bin/lsinitramfs
/usr/bin/lsipc
/usr/bin/lsirq
/usr/bin/lslocks
/usr/bin/lslogins
/usr/bin/lsmem
/usr/bin/lsmod
/usr/bin/lsns
/usr/bin/lsof
/usr/bin/lspci
/usr/bin/lspgpot
/usr/bin/lspower
/usr/bin/lstopo
/usr/bin/lstopo-no-graphics
/usr/bin/lsusb
/usr/lib/klibc/bin/ls
```

Использование updatedb

Во многих системах Linux есть "cron job" для периодического обновления базы. В случае если вызов locate вернул ошибку, то необходимо запустить updatedb от имени

рута для генерации поисковой базы.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ sudo -i
[sudo] password for kali:
┌──(root㉿kali)-[~]
└─# updatedb

┌──(root㉿kali)-[~]
└─#
```

7.4 slocate

В некоторых дистрибутивах вместо locate используется slocate. slocate подразумевает "безопасный locate". Он сохраняет информацию о правах доступа в поисковой базе так, что обычные пользователи не смогут увидеть директории, которые они и так не смогли бы увидеть. Используется slocate аналогично locate, однако вывод программы может отличаться в зависимости от того, какой пользователь ее запустил.

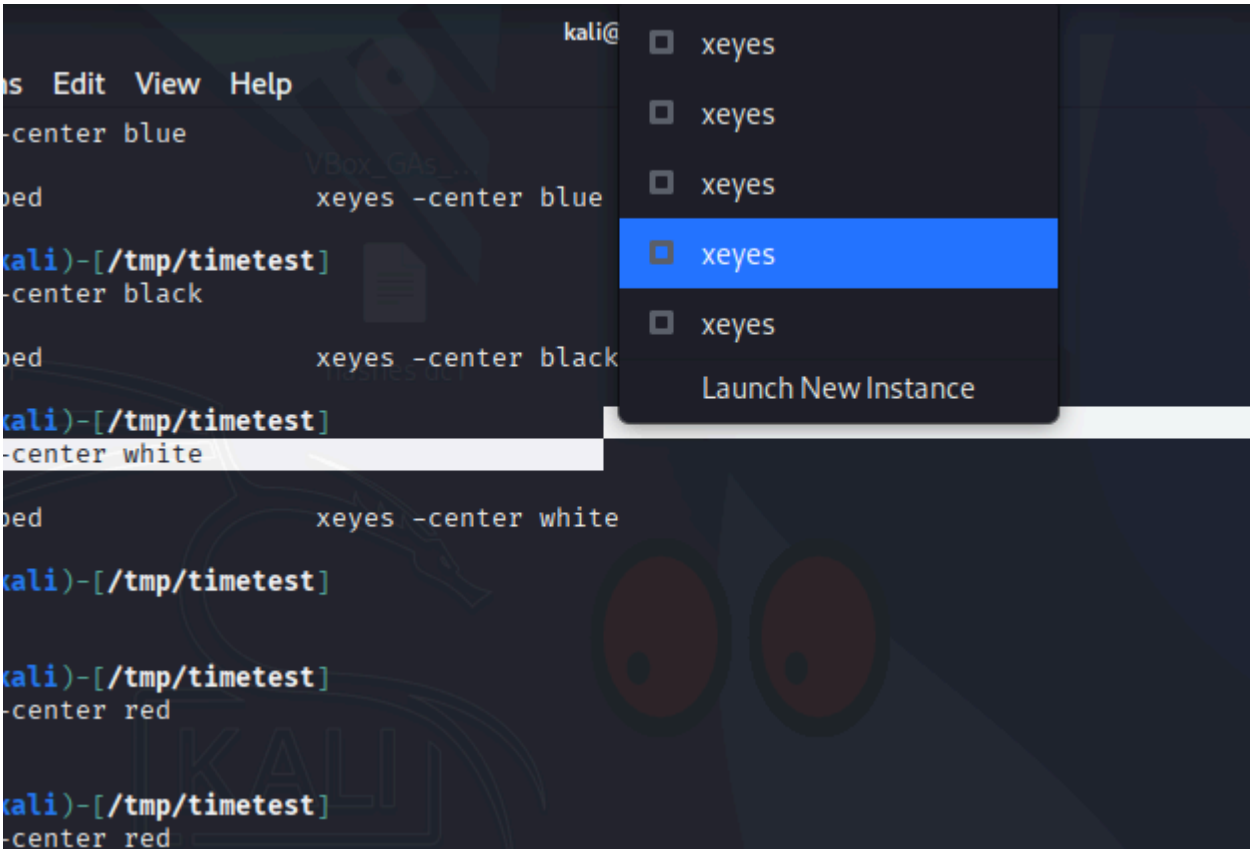
8. Управление процессами

Запуск xeyes

Для того, чтобы изучать какой-то процесс сначала надо его запустить. Для этого выполним команду xeyes -center red.
Далее появятся красные глаза, которые будут следить за курсором.

```
kali@kali: /tmp/timetest
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[/tmp/timetest]
└─$ sudo -i
[sudo] password for kali:
┌──(root㉿kali)-[~]
└─# updatedb

┌──(root㉿kali)-[~]
└─# logout

┌──(kali㉿kali)-[/tmp/timetest]
└─$ xeyes -center red
^Z
[1]+  Stopped                 xeyes -center red

┌──(kali㉿kali)-[/tmp/timetest]
└─$ xeyes -center blue
^[[A^[[B^Z
[2]+  Stopped                 xeyes -center blue

┌──(kali㉿kali)-[/tmp/timetest]
└─$ xeyes -center black
^Z
[3]+  Stopped                 xeyes -center black

┌──(kali㉿kali)-[/tmp/timetest]
└─$ xeyes -center white
^Z
[4]+  Stopped                 xeyes -center white

┌──(kali㉿kali)-[/tmp/timetest]
└─$

┌──(kali㉿kali)-[/tmp/timetest]
└─$ xeyes -center red
```

Чтобы вернуть приглашение и остановить тем самым процесс надо нажать Ctrl+C. Однако также можно вернуться к вводу команд и через Ctrl+Z, однако в таком случае процесс не будет убит и останется на фоне. Я так сделал примерно 5 раз, играясь с цветами, и получил вот это...:



В таком случае процесс не убивается, а лишь приостанавливается.

Чтобы разбудить процесс и запустить обратно применим команду fg.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ fg
xeyes -center red

^Z
[1]+  Stopped                 xeyes -center red

┌──(kali㉿kali)-[/tmp/timetest]
```

Далее продолжим его на фоне, используя bg.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ bg
[1]+ xeyes -center red &
```

Теперь красные от компа глазки следят за курсором через полупрозрачный терминал, а нам вернулась возможность вводить команды в оболочке.

8.2 Несколько фоновых процессов

Теперь посмотрим на наши процессы, запущенные в фоне, вернее на список таких процессов при помощи команды jobs.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ jobs -l
[1]- 108734 Running                 xeyes -center red &
[2]+ 108769 Running                 xeyes -center blue &

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

Самая левая колонка отображает номер процесса, который присваивается процессу при запуске. Плюс у второго задания означает, что если ввести fg, то в первую очередь он будет выведен. Также можно вывести на передний план другое задание, указав это в команде, например fg 1.
Следующая колонка - это идентификатор процесса, то есть PID. В данном случае PID появился из-за использования опции -l. В следующей колонке показано состояние процессов. "Running" говорит о том, что они выполняются. Командная строка каждого процесса находится в крайнем правом столбце.

9. Введение в сигналы

Для убийства, остановки или продолжения процесса в Linux используется специальная форма взаимодействия, называемая сигналы. Отправляя сигналы какому-то процессу, можно его завершить, остановить или еще что-нибудь сделать с ним. Это то, что на самом деле происходит при нажатии Ctrl+C и Ctrl+Z в Linux в терминале, а также при использовании fg и bg. То есть мы указываем bash на отправление процессу определенного сигнала. Сигналы также можно отправить с помощью команды kill. Например:

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ kill -s SIGSTOP 108769


[2]+  Stopped                 xeyes -center blue
┌──(kali㉿kali)-[/tmp/timetest]
└─$ jobs -l
[1]- 108734 Running                 xeyes -center red &
[2]+ 108769 Stopped (signal)        xeyes -center blue

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

Используя kill -s, можно отправить процессу любой сигнал. Можно убить, остановить или продолжить процесс соответствующим сигналом SIGINT, SIGSTOP, SIGCONT.

В случае, когда процесс игнорирует сигналы SIGTERM и SIGINT на завершение, например, вследствие того, что он как-то где-то застрял или уже остановлен, надо использовать уже не молоток, а кувалду, а именно SIGKILL. Процесс не может игнорировать SIGKILL.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ kill 108769

┌──(kali㉿kali)-[/tmp/timetest]
└─$ jobs -l
[1]- 108734 Running                 xeyes -center red &
[2]+ 108769 Stopped (signal)        xeyes -center blue

┌──(kali㉿kali)-[/tmp/timetest]
└─$ kill -s SIGKILL 108769

┌──(kali㉿kali)-[/tmp/timetest]
└─$ jobs -l
[1]- 108734 Running                 xeyes -center red &
[2]+ 108769 Killed                  xeyes -center blue

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

Терминал, в котором мы запускаем задания, называется терминалом управления заданиями. Некоторые шеллы (но не bash по умолчанию) отправляют сигнал SIGHUP фоновым заданиям, когда мы выходим, заставляя их принудительно завершиться. Для защиты процессов от этого можно использовать nohup когда мы запускаем процесс.

Для вывода списка процессов можно использовать ps. Дело в том, что jobs выводит только те процессы, которые были запущены в текущей сессии bash. Чтобы увидеть все процессы в системе надо использовать ps совместно с опциями a и x:

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ ps ax
  PID TTY      STAT   TIME COMMAND
    1 ?        Ss     0:01 /sbin/init splash
    2 ?        S      0:00 [kthreadd]
    3 ?        S      0:00 [pool_workqueue_release]
    4 ?        I<     0:00 [kworker/R-kvfree_rcu_reclaim]
    5 ?        I<     0:00 [kworker/R-rcu_gp]
    6 ?        I<     0:00 [kworker/R-sync_wq]
    7 ?        I<     0:00 [kworker/R-slub_flushwq]
    8 ?        I<     0:00 [kworker/R-netns]
   10 ?        I      0:00 [kworker/0:1-events]
   11 ?        I<     0:00 [kworker/0:0H-events_highpri]
   13 ?        I<     0:00 [kworker/R-mm_percpu_wq]
   14 ?        I      0:00 [rcu_tasks_kthread]
   15 ?        I      0:00 [rcu_tasks_rude_kthread]
   16 ?        I      0:00 [rcu_tasks_trace_kthread]
   17 ?        S      0:00 [ksoftirqd/0]
   18 ?        R      0:01 [rcu_preempt]
   19 ?        S      0:00 [rcu_exp_par_gp_kthread_worker/0]
   20 ?        S      0:00 [rcu_exp_gp_kthread_worker]
   21 ?        S      0:00 [migration/0]
   22 ?        S      0:00 [idle_inject/0]
   23 ?        S      0:00 [cpuhp/0]
   24 ?        S      0:00 [cpuhp/1]
   25 ?        S      0:00 [idle_inject/1]
   26 ?        S      0:00 [migration/1]
   27 ?        S      0:00 [ksoftirqd/1]
   29 ?        I<     0:00 [kworker/1:0H-events_highpri]
   31 ?        I      0:00 [kworker/u8:1-flush-8:0]
```

Команда дает слепок всего, что сейчас работает в системе, поэтому список такой длинный (на скрине только часть, а всего их там 200+). Без опции ax мы увидели бы только тех процессов, которые принадлежат нам, то есть только те, которые есть в

управляющем терминале.
Команда ps x покажет все процессы, которых даже нет в нашем терминале управляющем.
Команда ps a покажет список процессов из терминалов всех пользователей.

Опция --forest позволит легко просмотреть иерархию процессов и даст представление о том, как разные процессы в системе взаимосвязаны между собой. Если один процесс запускает второй, то запущенный будет его потомком. В выводе --forest, родители будут слева, а потомки справа.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ ps x --forest
    PID TTY      STAT   TIME COMMAND
    860 ?        Ssl    0:01 xfce4-session
   1009 ?        Sl     0:26  \_ xfwm4
   1046 ?        Sl     0:01  \_ xfsettingsd
   1052 ?        Sl     0:03  \_ xfce4-panel
   1070 ?        Sl     0:00  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1074 ?        Sl     0:25  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1075 ?        Sl     0:00  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1076 ?        Sl     0:28  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1077 ?        Sl     0:00  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1078 ?        Sl     0:00  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1079 ?        Sl     0:03  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1081 ?        Sl     0:00  |   \_ /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_
   1060 ?        Sl     0:00  \_ Thunar --daemon
   1066 ?        Sl     0:01  \_ xfdesktop
   1141 ?        Sl     0:00  \_ /usr/libexec/polkit-mate-authentication-agent-1
   1142 ?        Sl     0:00  \_ /usr/bin/python3 /usr/bin/blueman-applet
   1146 ?        Sl     0:00  \_ /usr/libexec/geoclue-2.0/demos/agent
   1147 ?        Sl     0:00  \_ xiccd
   1153 ?        Sl     0:01  \_ xfce4-screensaver
   1159 ?        S      0:00  \_ /usr/bin/python3 /usr/share/system-config-printer/applet.py
   1161 ?        Sl     0:00  \_ xfce4-power-manager
   1189 ?        Sl     0:00  \_ nm-applet
   2185 ?        Rl     0:09 /usr/bin/qterminal
   2191 pts/0    Ss     0:00  \_ /bin/bash
 108734 pts/0    S      0:00      \_ xeyes -center red
 126438 pts/0    R+     0:00      \_ ps x --forest
```

Опции u и l можно использовать в любой комбинации с опциями a, x с целью получения более подробной информации о процессах.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ ps au
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         678  0.4  5.0 354148 102360 tty7    Ssl+ 07:38   1:03 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var
root         680  0.0  0.1   8160  2588 tty1     Ss+  07:38   0:00 /sbin/agetty -o -- \u --noreset --noclear -
kali        2191  0.0  0.2   9184  5920 pts/0    Ss   07:40   0:00 /bin/bash
kali      108734  0.0  0.2   9368  4772 pts/0    S    11:20   0:00 xeyes -center red
kali      127152  0.0  0.2   9532  4188 pts/0    R+   11:58   0:00 ps au

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ ps al
F   UID     PID    PPID PRI  NI    VSZ   RSS WCHAN  STAT TTY        TIME COMMAND
4     0     678     665  20   0 354148 102360 -     Ssl+ tty7       1:03 /usr/lib/xorg/Xorg :0 -seat seat0 -aut
4     0     680       1  20   0   8160  2588 -      Ss+  tty1       0:00 /sbin/agetty -o -- \u --noreset --nocl
0  1000    2191    2185  20   0   9184  5920 do_wai Ss   pts/0      0:00 /bin/bash
0  1000  108734    2191  20   0   9368  4772 do_sys S    pts/0      0:00 xeyes -center red
0  1000  127241    2191  20   0   9564  4072 -      R+   pts/0      0:00 ps al

┌──(kali㉿kali)-[/tmp/timetest]
```

9.1 Использование top

Если мы обнаруживаем, что запускаем ps несколько раз подряд, пытаясь рассмотреть происходящие изменения, возможно нам стоит воспользоваться top. Программа top будет отображать постоянно обновляющийся список процессов, наряду в с другой полезной информацией.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ top
top - 12:00:16 up  4:21,  1 user,  load average: 0.14, 0.05, 0.01
Tasks: 161 total,   1 running, 160 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.7 us,  1.7 sy,  0.0 ni, 96.5 id,  0.2 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1973.7 total,    628.5 free,    814.0 used,    710.8 buff/cache
MiB Swap:    953.7 total,    953.7 free,      0.0 used.   1159.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    678 root      20   0  354148 102360  54888 S   2.0   5.1   1:04.41 Xorg
   2185 kali      20   0  572876  58356  48696 S   1.0   2.9   0:10.03 qterminal
   1009 kali      20   0  896544 134020  85308 S   0.7   6.6   0:26.55 xfwm4
   1074 kali      20   0  297160  51920  23296 S   0.7   2.6   0:25.46 wrapper-2.0
 108734 kali      20   0    9368   4772   4388 S   0.7   0.2   0:00.68 xeyes
    988 kali      20   0  168744   7612   6972 S   0.3   0.4   0:00.94 at-spi2-registr
   1052 kali      20   0  507320  44964  33520 S   0.3   2.2   0:03.79 xfce4-panel
      1 root      20   0   24020  14808  10680 S   0.0   0.7   0:01.06 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.51 kworker/0:1-events
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     16 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.52 ksoftirqd/0
     18 root      20   0       0      0      0 I   0.0   0.0   0:01.68 rcu_preempt
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
```

У каждого процесса есть свое значение приоритета, которое Linux использует для разделения времени CPU. Можно указать приоритет при запуске процесса с помощью команды nice. Например nice -n 10 oggenc /tmp/song.wav.
По умолчанию процессы запускаются с приоритетом 0, поэтому установка приоритета 10 значит, что он будет давать больше времени поработать другим процессам.

9.2 renice

Команда renice нужна для изменения приоритета у уже работающего процесса.

10. Обработка текста

10.1 Перенаправление

Помимо перенаправления вывода в файл можно также использовать каналы в оболочке, то есть пайпы, которые позволят передавать вывод одной команды на вход другой.
Например:

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ echo "hi there" | wc
      1       2       9

┌──(kali㉿kali)-[/tmp/timetest]
└─$
```

Сначала фраза подается в echo, после чего она подается в вход в wc, которая считает количество строк, слов и символов.

10.2 Распаковывающийся канал

Для разархивирования и распаковки можно использовать bzip2 и tar, но недостаток этих команд заключается в том, что создается промежуточный разархивированный файл на диске.

```
$ bzip2 -d linux-2.4.16.tar.bz2
$ tar xvf linux-2.4.16.tar
```

tar может читать данные напрямую со своего входа вместо указанного файла, мы можем получить тот же результат, используя пайп:

```
$ bzip2 -dc linux-2.4.16.tar.bz2 | tar xvf -
```

10.3 Канал подлиннее

Вот еще пример пайпа:
cat myfile.txt | sort | uniq | wc -l
В данном случае мы используем cat для отправки содержимого myfile команде sort. Команда sort сортирует содержимое файла в алфавитном порядке и передает дальше, то есть на вход в команду uniq, которая в свою очередь удаляет повторяющиеся строки и отправляет свой выход на вход в wc -l. wc -l будет выводить только количество строк, а количество слов и символов не выводятся.

11. Буря обработки текста

Далее приступим к рассмотрению команд Linux, которые предназначены для стандартной обработки текста.

- echo - печатает свои аргументы на терминал.
- echo -e - включить в вывод управляющие последовательности, например echo -e 'foo
  foo' напечатает foo, после чего перейдет на новую строку, а затем снова напечатает foo

- echo -n - запрещает добавлять echo символ новой строки в конец вывода
- cat - выводит текст из указанного файла в терминал. Удобно использовать в качестве первой команды для пайпа
- sort - выводит содержимое файла, указанного в командной строке, в алфавитном порядке
- uniq - принимает уже отсортированный файл или поток данных (через пайп) и удаляет повторяющиеся строки
- wc - выводит количество строк, слов и символов из входного потока
- head - выводит первые 10 строк текста из файла или входного потока
- head -n - вывод только строк, сколько будет указано, начиная с начала файла
- tail - вывод 10 строк текста с конца потока или файла
- tail -n - вывод нужного количества строк с конца потока данных
- tac - тоже, что и cat, но печатает строки в обратном порядке, то есть с конца в начало
- expand - конвертирует входные символы табуляции в пробелы
- expand -t - expand с указанием размера табуляции
- unexpand - конвертирует входные пробелы в символы табуляции. Опция -t для указания размера табуляции
- cut - извлечение из входного файла или потока полей, разделенных указанным символом. Например: echo 'abc def ghi jkl' | cut -d '' -f2,2
- nl - добавление к каждой входной строке номера
- pr - разбивает файл на страницы и нумерует их
- tr - трансляция (преобразование) символов. Может использоваться для отображения определенных символов во входном потоке на заданные символы в выходной поток
- sed - потоко-ориентированный текстовый редактор
- awk - искусственный язык построчного разбора и обработки входного потока по заданным шаблонам
- od - представление входного потока в восьмеричном, шестнадцатеричном и тп форматах
- split - разделение больших файлов на несколько небольших более управляемых частей
- fmt - выполнение переноса длинных строк текста. Сейчас уже не особо полезна, тк эта возможность встроена в большинство текстовых редакторов
- paste - принимает на вход 2+ файлов в качестве входных данных и объединяет их построчно, после чего выводит результат
- join - похожа на paste. Объединяет 2 файла по общему полю (по умолчанию первое поле в каждой строке)
- tee - печатает входные аргументы в файл и на экран одновременно

Перенаправление

Как и > в командной строке, мы можем использовать < для перенаправления файла на вход команде. Для многих команд можно просто указать имя файла.
Bash и другие шелы поддерживают концепцию "herefile", благодаря чему мы можем давать входные данные команды в виде набора строк с последующей командой, означающей окончание ввода последовательности значений.
Символы >> позволяют добавить вывод в файл, а не перезаписывать его каждый раз, как это делает >.

```
$ echo Hi > myfile
$ echo there. > myfile
$ cat myfile
there.
```

Уупс! Мы потеряли часть с «Hi»! А вот что мы имели ввиду:

```
$ echo Hi > myfile
$ echo there. >> myfile
$ cat myfile
Hi
there.
```

Так то лучше!

12. Модули ядра

12.1 uname

umane - предоставляет информацию о системе.

```
┌──(kali㉿kali)-[/tmp/timetest]
└─$ uname
Linux

┌──(kali㉿kali)-[/tmp/timetest]
└─$ sudo -i
[sudo] password for kali:
┌──(root㉿kali)-[~]
└─# uname
Linux

┌──(root㉿kali)-[~]
└─#
```

Можно использовать опции к команде:

- -s - имя ярда
- -n - имя хоста
- -r - релиз ядра
- -v - версия ядра
- -m - архитектура
- -p - процессор
- -i - платформа
- -o - операционная система
- -a - вся информация о системе

```
┌──(root㉿kali)-[~]
└─# uname -a
Linux kali 6.12.33+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25) x86_64 GNU/Linux
```

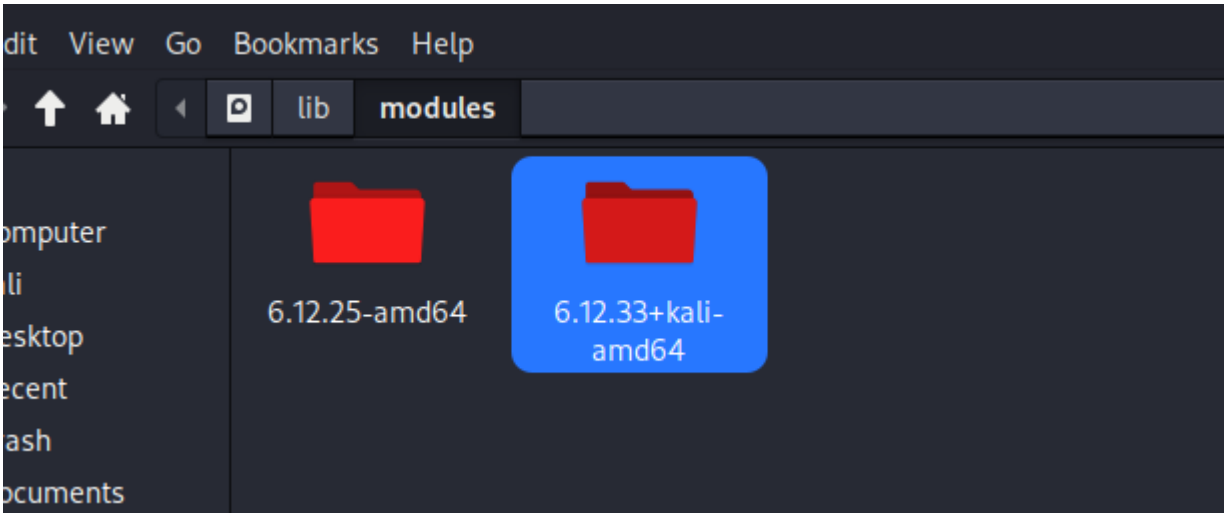
12.2 Релиз ядра

Для начала узнаем текущий релиз ядра:

```
┌──(root㉿kali)-[~]
└─# uname -r
6.12.33+kali-amd64

┌──(root㉿kali)-[~]
└─# sS
```

Теперь посмотрим в директорию /lib/modules. Здесь мы увидим каталог с таким же

названием.



12.3 Ядро

Ядро Linux - это кусок кода, который напрямую взаимодействует с нашим железом и абстрагирует от него обычные программы. Благодаря ядру текстовому редактору не надо беспокоиться о том, на какой диск он производит запись. Редактор просто записывает в файловую систему, а об остальном позаботится ядро.

У ядра есть модули. Это части ядра, которые сохраняются на диске в специальном формате. По наше команде они погружаются в работающее ядро и добавляют в него новую функциональность.
Поскольку модули ядра загружаются по требованию, мы можем иметь ядро, которое поддерживает дополнительную функциональность, которая в обычном состоянии выключена и недоступна. Зачастую эти модули загружаются автоматически, например для поддержки диковинной файловой системы или устройства, которое мы редко используем.

Модули ядра позволяют добавлять какие-то возможности в ядро.
Для просмотра загруженных модулей можно использовать команду lsmod.

```
└─# lsmod
Module                  Size  Used by
snd_seq_dummy          12288  0
snd_hrtimer            12288  1
snd_seq               110592  7 snd_seq_dummy
snd_seq_device         16384  1 snd_seq
rfkill                 40960  2
qrtr                   57344  2
vboxsf                 45056  0
sunrpc                872448  1
binfmt_misc            28672  1
snd_intel8×0           49152  1
snd_ac97_codec        196608  1 snd_intel8×0
intel_rapl_msr         20480  0
ac97_bus               12288  1 snd_ac97_codec
intel_rapl_common      53248  1 intel_rapl_msr
snd_pcm               188416  2 snd_intel8×0,snd_ac97_codec
snd_timer              53248  3 snd_seq,snd_hrtimer,snd_pcm
snd                   151552  10 snd_seq,snd_seq_device,snd_intel8×0,snd_timer,snd_ac97_codec,snd_pcm
soundcore              16384  1 snd
pcspkr                 12288  0
vboxguest              53248  6 vboxsf
ac                     16384  0
joydev                 24576  0
sg                     45056  0
evdev                  28672  10
efi_pstore             12288  0
configfs               69632  1
nfnetlink              20480  1
vsock_loopback         12288  0
```

```
особенность модулей ядра – возможность добавлять необходимую
функциональность в ядро и включать ее прямо в запущенное ядро, без
перезагрузки.
```

Информация о зависимостях между модулями со временем становится не актуальной. Для обновления этой информации можно использовать depmod -a. Эта команда просканирует модули из папки с ними (обычно это /lib/modules) и проверит так называемые "symbols" внутри модулей.

Для того, чтобы подгрузить модуль в ядро можно использовать команду insmod и указать ей полный путь к модулю, который мы хотим загрузить.
Также можно использовать команду modprobe. В отличие от insmod, modprobe автоматически загружает все необходимые зависимости для данного модуля.

Также есть команда rmmod, которая в отличие от modprobe, выгружает модуль.

Для того, чтобы узнать информацию о модуле можно использовать modinfo:

```
# modinfo fat

filename:    /lib/modules/2.4.20-gaming-r1/ker
description: <none>
author:      <none>
license:     "GPL"
```

12.4 Структура modules.conf

Файл modules.conf как правило содержит настройки.
Многие дистрибутивы генерируют этот файл автоматически, например в директории /etc/modules.d.

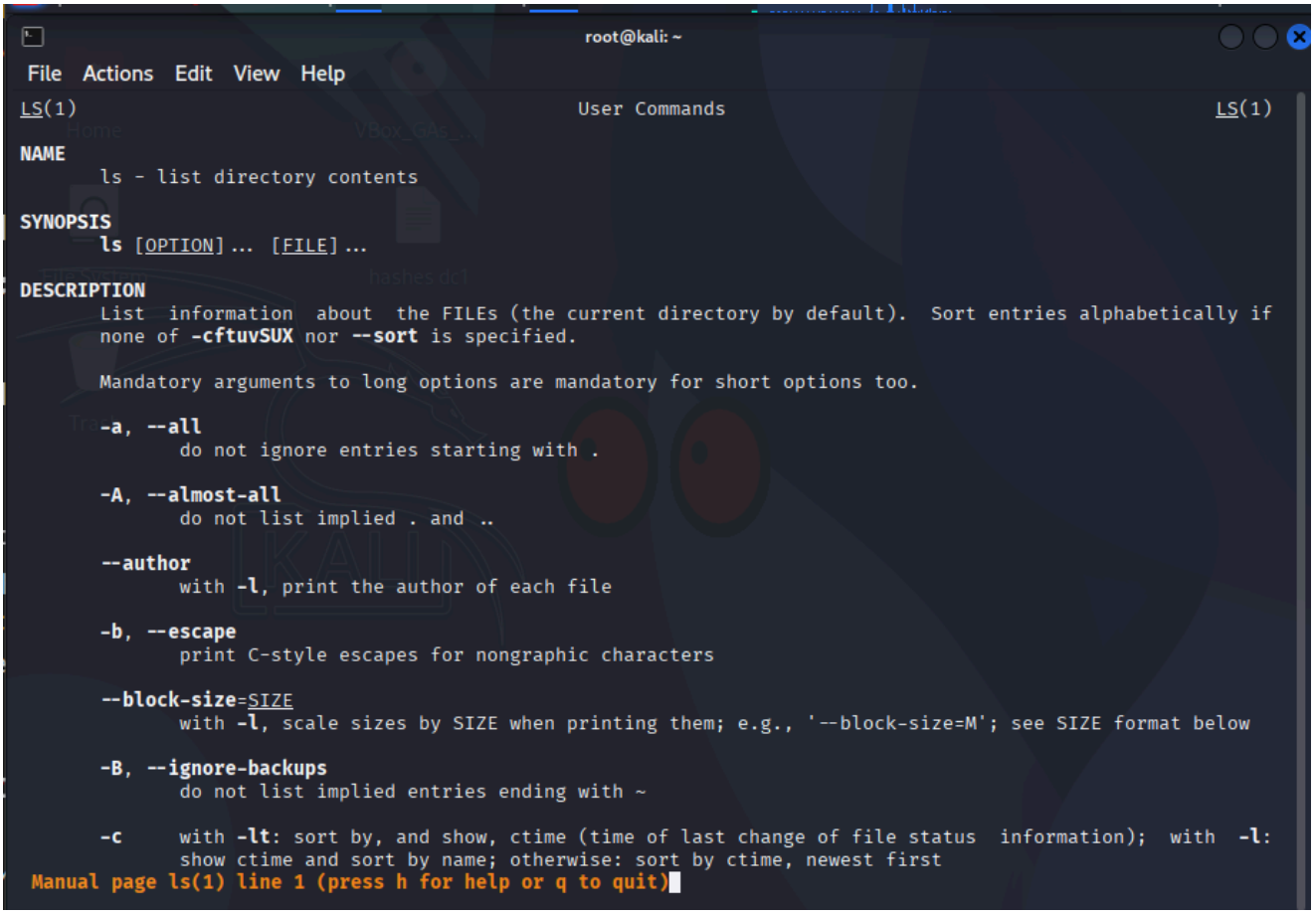
13. Системная и сетевая документация

13.1 Типы системной документации в Linux

Существует 3 основных источника документации в Linux системах: страницы руководств (маны), info-страницы, поставляемая с приложениями документация в /usr/share/doc.

Страницы руководств

Страницы руководств, или "man-страницы" - классическая форма справочной документации в UNIX и Linux. В идеале можно найти ман для любой команды, но на практике они есть не всегда.
Для доступа к ману надо просто ввести man и запрос. Запустится программа-пейджер (просмотрщик, обычно это less или more). Для закрытия программы надо как правило нажать q.
Например посмотрим ман для ls:



Файлы, содержащие информацию манов хранятся в /usr/share/man либо /usr/man. В этой директории можно найти страницы руководств, упорядоченные по следующим секциям:

man1	пользовательские программы
man2	системные вызовы
man3	библиотечные функции
man4	специальные файлы
man5	форматы файлов
man6	игры
man7	другое

Некоторые тему существуют более чем в 1 секции. Для демонстрации этого используем команду whatis, которая показывает доступные маны по теме:

```
┌──(root㉿kali)-[~]
└─# whatis printf
printf (1)           - format and print data
printf (3)           - formatted output conversion

┌──(root㉿kali)-[~]
└─#
```

В данном случае он обращается к 1 и 3 секции.

Теперь мы можем вызвать нужный ман:

```
root@kali: ~
File  Actions  Edit  View  Help
printf(3)                    Library Functions Manual                    pr

NAME
       printf, fprintf, dprintf, sprintf, snprintf, vprintf, vfprintf, vdprintf, vsprintf, vsnprintf -
       ted output conversion

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <stdio.h>

       int printf(const char *restrict format, ...);
       int fprintf(FILE *restrict stream,
                   const char *restrict format, ...);
       int dprintf(int fd,
                   const char *restrict format, ...);
       int sprintf(char *restrict str,
                   const char *restrict format, ...);
       int snprintf(char str[restrict .size], size_t size,
                   const char *restrict format, ...);

       int vprintf(const char *restrict format, va_list ap);
       int vfprintf(FILE *restrict stream,
                   const char *restrict format, va_list ap);
       int vdprintf(int fd,
                   const char *restrict format, va_list ap);
       int vsprintf(char *restrict str,
                   const char *restrict format, va_list ap);
       int vsnprintf(char str[restrict .size], size_t size,
                   const char *restrict format, va_list ap);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

 Manual page printf(3) line 1 (press h for help or q to quit)
```

Можно использовать man -k в случае, когда трудной найти нужную man-страницу по теме.
При запуске man -k фактически за кулисами стартует apropos.
Также есть команды makewhatis, которая сканирует все страницы в нашей системе и составляет из них базу данных для whatis и apropos.
По умолчанию, программа man будет искать ман-страницы в /usr/share/man, /usr/local/man, /usr/X11R6/man, и быть может в /opt/man.

13.2 GNU info

Одно из ограничений страниц руководств - это то, что они не поддерживают гипертекст, а значит нельзя просто перейти к другому руководству из текущего.
GNU решили эту проблему, и теперь у каждой команды в системе есть info-страница с подробным руководством.
Можно прочитать такую страницу по команде info.

Также нужно учитывать, что большая часть программ поставляется с доп документацией в пдф, postscript и html. Поэтому есть смысл при поиске информации о какой-то программе или команде также искать в каталоге /usr/share/doc, где будет длинный список из всех приложений на системе. Поиск по этой документации зачастую приводит к находке очень ценной информации.

14. Linux Documentation Project

В дополнение к существующей документации, в интернете есть ряд ресурсов, посвященных Linux.
Linux Documentation Project (LDP) - это группа добровольцев, которые занимаются составлением полного набора свободной документации по Linux. Данный проект существует чтобы собрать разные части документации по Linux в определенном месте, где ее будет легко искать и использовать.

14.1 Обзор LDP

LDP состоит из нескольких разделов:

- Guides - (руководства) - большие, очень серьезные пособия, такие как The Linux Programmers Guide
- HOWTOs - помощь по конкретной теме, например DSL HOWTO
- FAQs - сборники ответов на часто задаваемые вопросы, например Brief Linux FAQ
- Man pages - помощь по конкретной команде (это те же маны, что в системе)

Также LDP предоставляет доступ к списку ссылок и ресурсов, таких как Linux Gazette, Linux Weekly News, а также к спискам рассылки и архивам новостей.

Списки рассылок - это вероятно самое важное средство взаимодействия разработчиков Linux. Зачастую проекты разрабатываются на большом расстоянии от участников между собой, даже может быть на противоположных сторонах земли. Списка рассылки это по сути метод взаимодействия между собой разработчиков, в котором каждый разработчик проекта может связаться со всеми остальными и вместе дискутировать посредством электронной почты. Один из самых известных списков рассылки - Linux Kernel Mailing List.
Также списки рассылок предоставляют возможность задать вопросы и получить ответы от знающих разработчиков и других пользователей. К примеру, часто некоторые дистрибутивы предоставляют список рассылки для новичков.

Новостные группы в интернете похожи чем-то на списки рассылки, однако основаны на другом протоколе, который называется NNTP (Network Nwes Transfer Protocol - сетевой протокол передачи новостей), а не на обмене электронной почтой.
Чтобы иметь возможность общаться через него надо установить NNTP-клиент. Основное преимущество здесь в том, что можно принять участие в дискуссии когда нам это нужно, а не постоянно, когда она летит нам на почту.

Сайты разных дистрибутивов зачастую предоставляют обновленную документацию, инструкции по установке, информацию о совместимости с оборудованием и другие средства поддержи.
Например:

1. Redhat Linux
2. Debian Linux
3. Kali Linux
4. Fedora Linux
5. SuSE Linux
6. Caldera
7. Turbolinux
8. Поставщики аппаратного и программного обеспечения

Сейчас также на сайтах производителей устройств также можно найти информацию о поддержке Linux, например:

1. IBM and Linux
2. HP and Linux
3. Sun and Linux
4. Oracle and Linux